COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Sep 2, 2026	**DATE SUBMITTED:**	Sep 3, 2026
NAME:	Bal, Kevin Ranz J.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Seatwork 7.2: Algorithm Analysis: Best-Case and Worst-Case Time Complexity

1. PROCEDURE OUTPUT

What procedures did you perform?

PROCEDURE:

Procedure that performed:

Instructions: Analyze each algorithm carefully. For every item, determine the following:

1. **Best-Case Time Complexity**
2. **Worst-Case Time Complexity**
3. **Brief explanation of how you arrived at your answers.**

Use asymptotic notation such as O(1), O(log n), O(n), O(n log n), O(n^2), etc.

1. Finding an Element	**for (int i = 0; i < n; i++) {
	if (arr[i] == target) {
	return i;
	}
	}**
		Best-Case Time Complexity
		- O(n)
		
		Worst-Case Time Complexity
		- O(1)
		Brief explanation
		- The for loop iterates the N times based on the targets elements want and returns it. If the target was on the first element of the index, it will only iterate once but if not it will iterate each element

		of the array based on N.
2. Printing All Elements	for (int i = 0; i < n; i++) { cout << arr[i] << endl; }	Best-Case Time Complexity - O(n) Worst-Case Time Complexity - O(n) Brief explanation - The for loop always iterates the N times based on the index number

3. Nested Loops	for (int i = 0; i < n; i++) {
	for (int j = 0; j < n; j++) {
	cout << i << " " << j << endl;
	}
	}
		Best-Case Time Complexity
		- O(n^2)
		Worst-Case Time Complexity
		- O(n^2)
		Brief explanation
		- The Outer for loop iterates the N times and the inner for loop of the outer loop N times which is T(n) = (N)(N) = O(n^2)

4. Conditional Processing	if (n > 100) {
	for (int i = 0; i < n; i++) {
	cout << arr[i];
	}
	} else {
	cout << "Small input";
	}
		Best-Case Time Complexity
		- O(1)
		Worst-Case Time Complexity
		- O(n)
		Brief explanation
		- The if condition checks if the n is greater than 100 and if the condition was right it will iterate the N times, but if the N is less than 100, it will skip the if condition.

5. Fixed Inner Loop	for (int i = 0; i < n; i++) {
	for (int j = 0; j < 10; j++) {
	cout << i << j;
	}
	}
		Best-Case Time Complexity
		- O(n)
		Worst-Case Time Complexity
		- O(n)
		Brief explanation
		- The outside for loop iterates N times based on the given N, and inside the inner for loop it will iterates ten times

6. Two Consecutive Loops	for (int i = 0; i < n; i++) {
	cout << arr[i];
	}
	for (int j = 0; j < n; j++) {
		Best-Case Time Complexity
		- O(n)
		Worst-Case Time Complexity
		- O(n)

	cout << arr[j] * 2; }	Brief explanation - The two for loop that has been both outside has been iterated N times both which will be T(n) = N + N = 2N which is the time complexity is O(n)

7. Dividing by Two	int x = n;
	while (x > 1) {
	x = x / 2;
	}
		Best-Case Time Complexity
		- O (1)
		Worst-Case Time Complexity
		- O(log2 n)
		Brief explanation
		- The x was in a condition where x is greater than 1 and if the condition has been met, it will iterate the x and divide it to 2 until the x value no longer greater than 1 that on the given condition, if the condition hasn't been met at first, it will only iterate once and gives a time complexity of O(n)

8. Nested Loop With Different Limits	for (int i = 0; i < n; i++) {
	for (int j = 0; j < i; j++) {
	cout << i << j; } }
		Best-Case Time Complexity
		- O(n)
		Worst-Case Time Complexity
		- O(n^2)
		Brief explanation
		- The i is set to zero or in a negative value, it will iterate by the N and since the inner for loop didn't iterate, it will iterate it by N^2 because the value is positive and a non-zero and the inner or nested loop iterates the outer for loop until it meets the condition

9. Early Termination	for (int i = 0; i < n; i++) {
	if (arr[i] == target) {
	cout << "Found";
	break;
	}
	}
		Best-Case Time Complexity
		- O(1)
		Worst-Case Time Complexity
		- O(n)
		Brief explanation
		- When the target element reaches the first element of the iteration it will display "Found" and the iteration will only happen

		once that gives O(1) else, the target will continue to iterate until the target it found

10. Two-Level Search	for (int i = 0; i < n; i++) {
	for (int j = 0; j < n; j++) {
	if (arr[i] == target) {
	break;
	}
	}
	}
		Best-Case Time Complexity
		- O(n)
		Worst-Case Time Complexity
		- O(n^2)
		Brief explanation
		- WHen the arr array index is matched / equal to the target, the inner for loop breaks and the outer for loop will continue until the condition is met and it gives a result of O(n), and if the condition of the arr and target hasn't been equal or found yet, the both loops will continue to iterate

11. Linear Loop With Logarithmic Inner Work	for (int i = 0; i < n; i++) {
	int x = n;
	while (x > 1) {
	x = x / 2; } }
		Best-Case Time Complexity
		- O(n log2 n)
		Worst-Case Time Complexity
		- O(n log2 n)
		Brief explanation
		- The outer for loop iterates by N times for each of the iteration and the inner for loop continues to divide by 2 until it no longer match to the condition of the value of x being greater than 1

12. Conditional Nested Loop	if (arr[0] == target) {
	cout << "Found";
	} else {
	for (int i = 0; i < n; i++) {
	for (int j = 0; j < n; j++) {
	cout << arr[i] + arr[j];
	}
	}
	}
		Best-Case Time Complexity
		- O(1)
		Worst-Case Time Complexity
		- O(n^2)
		Brief explanation
		- The best case is reached when the index value was immediately matches the target and only performs one execution of the program, but if not, it will iterate the inner

		and outer for loop that results of O(n^2)

13. Finding the Maximum	int max = arr[0];
	for (int i = 1; i < n; i++) {
	if (arr[i] > max) {
	max = arr[i];
	}
	}
		Best-Case Time Complexity
		- O(n)
		Worst-Case Time Complexity
		- O(n)
		Brief explanation
		- The for loop for the array is searching every element of the array from the start that start from index 0 which is the time complexity is O(n)

14. Three Nested Loops	for (int i = 0; i < n; i++) {
	for (int j = 0; j < n; j++) {
	for (int k = 0; k < n; k++) {
	cout << i + j + k;
	}
	}
	}
		Best-Case Time Complexity
		- O(n^3)
		Worst-Case Time Complexity
		- O(n^3)
		Brief explanation
		- There is a for loop outside and inside of it was another loop which is called nested loop and inside the nested loop there is an another loop which iterate each of them and giving time complexity of O(n^3)

15. Comprehensive Analysis	for (int i = 0; i < n; i++) {
	if (arr[i] == target) {
	return i;
	}
	int x = n;
	while (x > 1) {
	x = x / 2;
	}
	}
		Best-Case Time Complexity
		- O(n log n)
		Worst-Case Time Complexity
		- O(n log n)
		Brief explanation
		- If the index is matched to the target, it will continue to the logarithmic loop, and if not, it will iterate and perform the logarithmic operation until it meets the condition that gives a time complexity of O(n log n)

ILO A

Task Title & Task Objective

Screenshot of the code	
Output of the code	
Analysis	

ILO B

Task Title & Task Objective

Screenshot of the code	
Output of the code	
Analysis	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SUPPLEMENTARY:

ILO C

Task Title & Objective:

Screenshot of the code	
Output of the code	
Analysis	

Conclusion

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.